



저작자표시-비영리-변경금지 2.0 대한민국

이용자는 아래의 조건을 따르는 경우에 한하여 자유롭게

- 이 저작물을 복제, 배포, 전송, 전시, 공연 및 방송할 수 있습니다.

다음과 같은 조건을 따라야 합니다:



저작자표시. 귀하는 원저작자를 표시하여야 합니다.



비영리. 귀하는 이 저작물을 영리 목적으로 이용할 수 없습니다.



변경금지. 귀하는 이 저작물을 개작, 변형 또는 가공할 수 없습니다.

- 귀하는, 이 저작물의 재이용이나 배포의 경우, 이 저작물에 적용된 이용허락조건을 명확하게 나타내어야 합니다.
- 저작권자로부터 별도의 허가를 받으면 이러한 조건들은 적용되지 않습니다.

저작권법에 따른 이용자의 권리는 위의 내용에 의하여 영향을 받지 않습니다.

이것은 [이용허락규약\(Legal Code\)](#)을 이해하기 쉽게 요약한 것입니다.

[Disclaimer](#)

공학석사학위논문

Unity-ROS 기반 가상 시뮬레이션 환경을
통한 소형 무인보트 임베디드
시스템 구현

Implementation of a Small Unmanned Surface Vehicle Emb
edded System Using a Unity-ROS-Based Virtual
Simulation Environment

2025년 1월

국립창원대학교 대학원

문화융합기술협동과정

이 세 진

공학석사학위논문

Unity-ROS 기반 가상 시뮬레이션 환경을
통한 소형 무인보트 임베디드
시스템 구현

영문

Implementation of a Small Unmanned Surface Vehicle Emb
edded System Using a Unity-ROS-Based Virtual
Simulation Environment

지도교수 유 선 진

이 논문을 공학석사학위논문으로 제출함

2025년 1월

국립창원대학교 대학원
문화융합기술협동과정
이 세 진

이세진의 공학석사학위논문을 인준함

심사위원장 남 상 훈 (인)

심사위원 유 선 진 (인)

심사위원 오 범 석 (인)

2025년 1월

국립창원대학교 대학원

목 차

I. 서론

1.1 연구 배경 및 필요성	1
1.2 국내외 연구 현황	2
1.3 연구 목적 및 주요 내용	4
1.4 논문의 구성	4

II. 이론적 배경 및 시스템 구성

2.1 기존 연구사례 분석	5
2.2 3D 프린팅 활용한 선체 제작	7
2.3 센서 구성 및 ROS	8

III. 가상환경 구성 및 선박 제작

3.1 Unity 환경 구축	11
3.1.1 Unity 프로젝트 설정	11
3.1.2 가상 해양 환경 구성	12
3.2 가상센서	13
3.2.1 LIDAR 가상 센서	13

3.2.2 가상 카메라	14
3.2.3 가상 GPS 및 IMU	15
3.2.4 가상 쓰러스터	15
3.2.4 센서 데이터 통합	16
3.2.6 알고리즘 구현 및 적용	17
3.3 3D 프린터 선박 제작 및 임베디드 시스템 설치	19
3.3.1 테스트 시나리오 설정	19
3.3.2 테스트 수행 및 데이터 수집	21
IV. 임베디드 시스템 코드 테스트	
4.1 테스트 환경 설정 및 실행	24
4.2 테스트 결과 및 분석	26
V. 결론	

그 립 목 차

그림 1.	1
그림 2. 현대 아비커스 자율운행 보트,	3
그림 3. 쌍동선 구조	7
그림 4. ROS 통신 구성도	9
그림 5. Unity 가상 환경 화면	12
그림 6 . Unity(좌)-ROS(우) 연결된 LIDAR 화면	13
그림 7. Linux로 전송되고 있는 Unity의 가상카메라	14
그림 8. Unity에서 구현된 가상 쓰러스터 모델	15
그림 9. Rhino에서 제작된 무인보트(위)와 Unity 적용(아래)	19
그림 10. 무인선박 제작 과정	20
그림 11. 컨트롤 박스 제작 과정	21
그림 12. 코드 적용을 위한 노드 확인	22
그림 13. 실험을 위한 수조	24
그림 14. 웨이포인트 주행(위)와 주행 경로 비교(아래)	27
그림 15. 무인보트의 장애물 회피	28
그림 16. YOLO 적용(위)와 도킹 하는 장면	29
그림 17. 종합 임무 시나리오	30

표 목 차

<u>표 1. 테스트를 위한 시나리오</u>	25
<u>표 2. 테스트 결과</u>	26



국문 초록

최근 무인보트(USV, Unmanned Surface Vehicle) 기술은 해양 모니터링, 구조·수색, 안전관리 등 다양한 분야에서 주목받고 있으나, 실제 개발과 실험에 높은 비용과 물리적 제약이 따른다. 본 연구에서는 Unity 기반의 가상환경과 ROS2를 결합해, 무인보트 자율항해 알고리즘을 시뮬레이션 단계에서 빠르게 구현·검증한 뒤, 동일한 코드 구조를 실제 임베디드 시스템에 이식함으로써 개발 효율을 높이는 방법을 제안한다.

우선 기존 연구사례를 조사하여 소형 무인보트에 적합한 쌍동형(catamaran) 선체를 3D 프린팅으로 제작하고, LIDAR, 카메라, GPS, IMU 등 센서 구성을 검토하였다. Unity 환경에서는 Raycasting을 통한 가상 LIDAR, transform 정보를 활용한 GPS·IMU 모사, 카메라 RenderTexture를 이용한 영상 생성 등 가상 센서 모델을 구현하여, ROS2 메시지 형태로 발행하도록 설계하였다. 이를 통해 시뮬레이션에서 코드 디버깅과 알고리즘 보완이 용이해졌으며, 실제 선체·센서·보드(젯슨 나노)로 구성된 무인보트에 별다른 수정 없이 동일 코드를 적용할 수 있었다.

수조 실험 결과, GPS 기반 웨이포인트 주행, LIDAR 장애물 회피, 이미지 인식 도킹 등 개별 임무에서 약 80~90%의 성공률을 달성했고, 통합 임무를 수행할 때 노드 간 충돌이나 센서 노이즈 등으로 성공률이 다소 떨어지는 사례가 관찰되었다. 이는 현실 환경에서 발생하는 노이즈·지연 등을 시뮬레이션이 완전히 반영하지 못한 한계이지만, 초기 개발 위험과 비용을 낮추고 반복 실험을 안전하게 수행하는 측면에서 가상환경-현실 간 일관된 ROS2 프레임워크가 유용함을 확인하였다. 향후에는 대형 선박 적용, 해상 실험 확장, 딥러닝 기반 인식 기능 등 고도화를 통해 무인보트 기술의 범용성을 더욱 확장할 수 있을 것으로 기대된다.

키워드

USV, Autonomous Navigation, ROS2, Unity Simulation, 3D Printing,
Virtual simulation



I. 서론

1.1 연구 배경 및 필요성

최근 무인체(Unmanned System)에 대한 관심이 증가함에 따라 자율주행 기술은 자동차, 드론, 로봇, 선박 등 다양한 분야로 확산되고 있다. 특히 선박 분야에서는 높은 인건비와 안전 문제, 해양 환경 모니터링 등에 대한 요구가 맞물려 자율운항 선박 및 무인보트(Unmanned Surface Vehicle, USV) 기술이 급속히 발전하고 있다. 시장조사기관 어큐트마켓은 자율운항 선박 시장은 2021년 약 95조 원에서 2025년까지 180조 원대로 성장할 것으로 예상한다[1]. 정부는 2025년까지 점진적으로 부분 자율운항 기술의 연구 진행을 끝내고 2031년부터 완전히 자율화된 4단계 기술 도입을 계획함과 동시에 무인 원격 제어가 가능한 IMO 자율화 등급 3단계에 해당하는 기술의 상용화를 위한 실증과 근거 법률의 제정을 추진하고 있다[2].

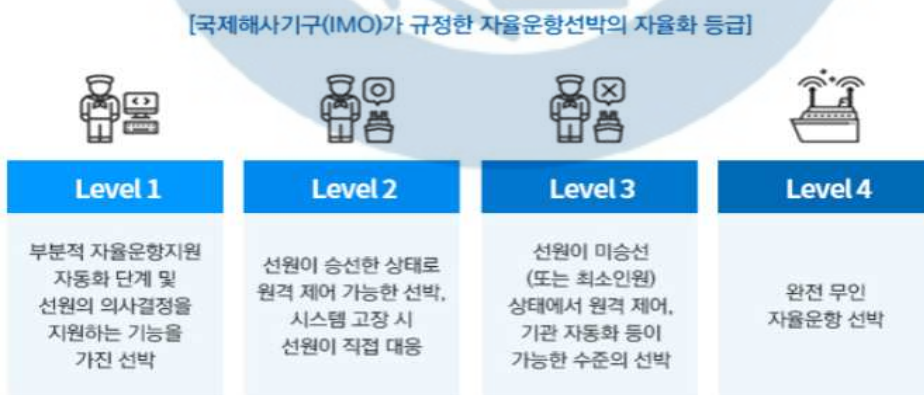


그림 1. 자율운항 4단계

이처럼 무인보트 기술의 산업적·기술적 잠재력이 크지만, 실제 무인보트의 개발 과정은 지상 차량이나 드론에 비해 훨씬 번거롭고 비용이 많이 든다. 무엇보다 실험 환경을 조성하기가 까다로운데, 물이 있는 장소를 구하거나 소규모 수조를 설치해도 공간 제약과 안전 문제가 발생하고, 반복 실험 시에는 선체와 장비를 옮겨 다녀야 하는 부담이 크다[3]. 무인보트 선체를 제작하고, LIDAR·카메라·GPS·IMU 등 다양한 센서를 배선해 통합한 뒤, 이를 실제 수면 위에 띄워 테스트하는 과정은 예상보다 훨씬 많은 시간을 소모한다. 센서 오작동이나 제어 오류를 발견할 때마다 다시 회수 후 수정해야 하므로, 개발자가 여러 차례 반복 실험을 진행하기가 쉽지 않다. 따라서 이러한 물리적 실험의 어려움을 보완하고, 무인보트 연구·개발 초기에 발생하는 높은 진입장벽을 낮추기 위해서는 가상환경에서의 검증을 적극 활용할 필요가 있다. 가상환경 시뮬레이션을 사용하면 실험 장소·시간의 제약을 크게 줄일 수 있고, 위험 부담 없이 자유롭게 시나리오를 반복 실행할 수 있어 개발 속도를 높일 수 있다.

1.2 국내외 연구 현황

해외에서는 무인보트의 개발과 관련하여 시뮬레이션 환경을 활용한 연구가 활발히 진행되고 있다. 특히 미국의 VRX(Virtual RobotX) 시뮬레이터는 오픈 소스 로봇 운영 체제인 ROS와 Gazebo 시뮬레이터를 기반으로 무인보트의 시뮬레이션 환경을 제공하고 있다. VRX는 가상 환경에서 센서 데이터와 다양한 시나리오를 구현하여 무인보트의 자율 향해 알고리즘을 개발하고 테스트하는 데 활용되고 있다[4]. 매년 VRX를 활용한 대회도 열릴 만큼 그 관심도도 높다. 하지만 VRX는 접근성이 제한적이거나, 복잡한 설정과 높은 이해도 요구로 인해 일반 개발자나 연구자들이 활용하기 어렵다.



그림 2. 현대 아비커스 자율운행 보트

국내에서는 현대중공업의 자회사인 아비커스(Avikus)가 무인보트 및 자율운항 선박 기술 개발에 앞장서고 있다. 아비커스는 세계 최초로 대형 선박의 완전 자율운항을 성공적으로 수행하였으며, 관련 기술을 지속적으로 개발하고 있다[5]. 그러나 이들의 연구 내용과 기술은 일반인들에게 공개되어 있지 않아 접근성이 제한적이다. 이는 학계나 중소기업, 개인 개발자들이 해당 기술을 활용하거나 발전시키는 데 어려움을 초래하고 있다.

국내의 학계 및 연구기관에서도 무인보트 시뮬레이션과 실제 구현에 대한 연구와 자율운항보트 경진대회(KABOAT) 같은 대회가 진행되고 있다. 이런 연구와 대회등을 통하여 무인보트의 자율 항해 알고리즘 개발과 시뮬레이션을 수행하고 있지만, 실제 선체 제작과 다양한 센서의 통합, 그리고 현실과 유사한 가상환경 구축을 통한 코드 검증까지 아우르는 종합적인 연구는 부족한 실정이다.

1.3 연구 목적 및 주요 내용

본 연구의 목적은 현실과 유사한 가상환경을 구축하고, 해당 가상환경에서 무인보트 제어 코드를 검증한 뒤, 이를 실제 선체와 센서 시스템에 적용하여 유의미한 동작을 확인하는 데 있다. 가상환경을 통해 초기 개발 비용과 물리적인 환경 제약을 줄이고, 센서·제어 알고리즘을 반복 시험할 수 있도록 한다. 이후 가상환경에서 충분히 테스트된 코드와 구조를 실제 무인보트에 그대로 이식함으로써, 현실 세계에서 해당 알고리즘이 제대로 작동하는지 확인한다.

Unity 엔진을 활용하여 실제 선체 모델을 바탕으로 가상 선체를 구현하고, LIDAR, 카메라, GPS, IMU 등 가상의 센서 모델도 구현하였다. Unity와 ROS 2를 결합하여, 가상환경에서의 보트 동작과 센서 데이터를 실시간으로 처리·검증할 수 있도록 구성하였다. 가상환경에서 이러한 코드 검증이 끝난 후에는, 실제 선체를 3D 프린터로 제작하고, 동일한 센서(LIDAR, 카메라, GPS, IMU)를 탑재한 선박을 제작한다. 최종적으로, 가상환경에서 테스트를 마친 ROS2 노드를 임베디드 보드에 이식하여 물리 환경에서의 자율주행 및 센서 처리를 수행함으로써, 시뮬레이션 결과가 현실 세계에서 얼마나 유의미하게 유지되는지를 검증하고자 한다.

1.4 논문의 구성

본 논문은 다음과 같이 구성된다. 제2장(이론적 배경 및 시스템 구성)에서는 먼저 국내외 기존 연구사례를 분석하여 무인보트 분야에서 수행된 다양한 실험과 적용 기술을 살펴 본 후 본 연구에서 이용할 선체 형태와 주행에 필요한 센서와 시스템을 참고하고 ROS2 구조와 Unity 시뮬레이션 구

축 방법을 제시한다. 이어서 제3장에서는 가상 센서 구현 및 가상 선박 움직임 제어를 검증한 뒤, 실제 3D 프린터 선체와 젯슨 나노(Jetson Nano) 보드를 결합해 무인보트를 구성하는 과정을 다룬다. 제4장에서는 임베디드 시스템 상에서 GPS, LIDAR, 카메라 등을 이용한 네 가지 테스트 시나리오를 진행하여 결과를 분석하고, 마지막 제5장에서는 연구 성과와 한계를 정리함하고 센서 노이즈 모델링, 딥러닝 인식 강화 등 향후 연구 방향을 논의한다.

II. 이론적 배경 및 시스템 구성

2.1 기존 연구사례 분석

선박의 충돌 회피는 과거부터 다양한 보조 시스템이 개발되어 왔지만, 최종적으로는 조종 관계자의 경험과 판단에 크게 의존하는 경우가 많았다고 보고되어 있다. 특히 중소형 선박의 경우, 충돌 회피를 보조하는 첨단 시스템을 설치하기가 쉽지 않고 조종자의 경험도 충분치 않아 충돌 사고가 계속 발생하는데, 이 중 상당수는 인간의 오류로 드러났다[6]. 이러한 문제를 해결하기 위해 선박 충돌 회피 알고리즘에 자율운항 기술을 적용하고자 하는 연구가 활발히 이루어지고 있으며, 안전성 확보를 위해 실험대신 동역학 모델 기반 시뮬레이션을 활용하는 사례가 많다. 실제 선박 운항 상황은 위험 부담이 크고 반복 실험이 쉽지 않기 때문에, 충돌 회피 알고리즘을 개발하고 검증하는 데 시뮬레이션이 중요한 도구로 인식되는 것이다.

최근 연구들은 주로 ROS 환경에서 센서들을 연동하고, 자율운항 알고리즘을 개선하거나 검증하는 데 초점을 맞추고 있다. 예를 들어, 어떤 연구에서는 LIDAR 등 다양한 센서를 보트에 탑재하고 ROS를 이용해 자율운항 알고리즘을 구현한 뒤 실제 시험으로 성능을 검증함으로써, ROS 기반 소프트웨어 구조가 하드웨어 센서와 유기적으로 연동되는 장점을 강조하였다[7]. 또 다른 연구에서는 저가형 LiDAR 센서를 적극 활용하여 부드러운 선회 환경을 특징으로 하는 충돌 회피 알고리즘을 개발했는데, 저가 센서만으로도 안정적인 회피가 가능함을 입증했다[8]. 이 밖에도 협수로(狹水路)에서의 충돌 회피를 위해 비용 함수를 활용하여 곡선 좌표계를 구축하고 시뮬레이션으로 알고리즘을 검증한 사례[9], 또는 ROS 기반 Gazebo 시뮬레이터에서 2D LiDAR와 무인선박을 구현해 충돌 회피 성능을 시험한 연구 등도 보고되고 있다[10]. 이러한 연구들 대부분은 ROS상에서 센서 정보를 받아 자율운항 알고리즘을 테스트하는 데 집중하며, 실제 임베디드 시스템 배포나 하드웨어 최적화보다는 알고리즘 완성도와 시뮬레이션 결과에 초점을 맞추는 양상을 보인다.

가상환경을 구현하는 시뮬레이션 도구로는 Unity를 사용하는 사례가 늘고 있는데, Unity가 강력한 그래픽 엔진과 물리 엔진을 제공하므로, 현실적인 시각·물리 환경을 구축하는 데 유리하다는 평가가 이어져왔다[11]. 어떤 연구에서는 Unity와 Gazebo를 병행하여 자율주행 자동차 시뮬레이션을 구현하고 비교함으로써, Unity가 높은 수준의 그래픽 표현과 사용자 친화성을 지닌다고 보고하였다[12]. 또 다른 연구에서는 ROS와 결합한 USV-UAV 시뮬레이션 환경을 구성하여 다중 무인체 시나리오를 폭넓게 테스트하기도 했는데, 이를 통해 다양한 센서 데이터와 충돌 회피 알고리즘을 실험할 수 있음을 입증했다. 선박해양공학 분야에서도 Unity가 자동 소화 시스템 제어 알고리즘이나 배관 경로 설계에 적용된 사례가 있으나, 주로 강화학습

환경 제공에 집중했을 뿐, 선박 운항 자체를 시뮬레이션하는 연구는 상대적으로 드문 편이다[13][14].

이처럼 다양한 연구에서 ROS와 Unity가 센서·알고리즘 검증에 유용하다고 제시되었음에도, 실제 선체를 제작하는 과정에서 가상환경에서 검증한 뒤 임베디드 시스템으로 이식하는 시도는 충분히 보고되지 않았다. 기존 연구들은 ROS 환경에서 센서 기반 주행 알고리즘을 개선하거나, Unity 시뮬레이션으로 충돌 회피 로직을 시험하는 정도에서 마무리되는 양상이 대부분이었다. 따라서 본 연구에서는 ROS와 Unity를 결합하여 소형 무인보트에 적합한 시스템을 설치하고, 이를 실제 임베디드 시스템에까지 적용하는 프로세스를 제시하고자 한다.

2.2 3D 프린팅 활용한 선체 제작

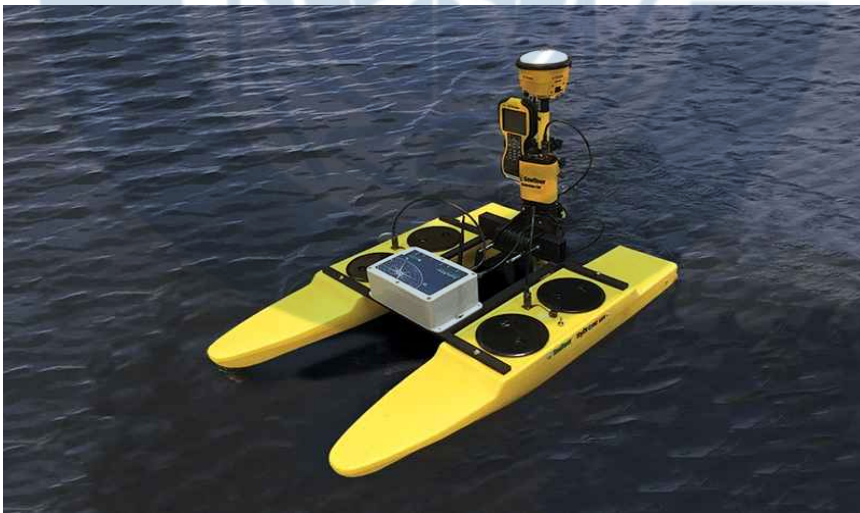


그림 3. 쌍동선 구조

본 연구에서는 기존 연구사례를 참고하여, 소형 무인보트의 안정성과 기동

성을 동시에 확보하기 위해 쌍동형(catamaran) 선박 구조를 선택하였다[15][16]. 쌍동형 선체는 양쪽에 두 개의 좁은 선체가 평행하게 배치되어 있어, 단동형(monohull)에 비해 선체 폭이 넓고, 비교적 작은 파도나 수면 요동에도 안정적인 부력을 유지할 수 있는 장점이 있다. 또한 선체 간 수면저항이 분산되어, 속도와 선회 성능 면에서도 유리하다는 평가가 여러 연구에서 제시되어 왔다.

쌍동형 보트 구조를 구현하기 위해, 본 연구는 3D 프린팅 기법을 적극 활용하였다. 3D 프린터를 사용하면 선체 부품을 개별적으로 출력한 뒤 결합할 수 있어, 설계 변경이나 부분 보수에 유연하게 대처할 수 있다. 또한 제작 비용과 시간을 절감하고, 실험용 소형 선체를 여러 가지 형태로 시도해볼 수 있다는 장점이 있다[17]. 3D 프린팅으로 출력한 선체는 내부식성과 방수성을 강화하기 위해 표면 코팅 및 접합부 실링 작업을 수행하였다. 이런 과정을 통해 소형 쌍동형 선체가 가진 이점을 최대한 살리면서, 향후 센서나 임베디드 보드를 배치하기에 충분한 공간과 구조적 안정성을 확보했다.

본 연구가 시도하는 3D 프린팅 기반 쌍동형 선체 제작은 중소 규모 연구팀이나 교육 현장 등에서 비교적 적은 예산으로 무인보트 프로토타입을 구성할 수 있다.

2.3 센서 구성 및 ROS-Unity 연동

본 연구에서 설계한 소형 무인보트에는 자율항해에 필수적인 LIDAR, GPS, IMU, 카메라 등 주요 센서들이 탑재된다. LIDAR(Light Detection and Ranging)는 360도 회전을 통해 주변 장애물까지의 거리 정보를 획득하며, 무인보트가 물 위에서 충돌을 예방하고 경로를 계획할 수 있도록 한다. GPS

(Global Positioning System)는 보트의 전역 좌표(위도·경도·고도)를 실시간으로 제공하여 목표 지점까지의 웨이포인트 주행과 속도 제어에 활용된다. IMU(Inertial Measurement Unit)는 3축 가속도·자이로·지자기 센서 등을 포함해 보트의 자세(Roll, pitch, yaw)와 가속도, 각속도 등을 측정하고, 이 데이터를 GPS와 결합해 위치·속도를 정교하게 추정한다. 카메라는 전방 시야 정보를 획득해 객체나 부표 등을 인식하는 데 쓰이며, 필요 시 딥러닝 기반 객체 탐지 알고리즘과 결합할 수도 있다. 이러한 센서들은 모두 보드(예: 젯슨 나노 또는 젯슨 오린)에서 각각의 드라이버 노드가 구동되도록 설정하여, ROS2 메시지 형태로 데이터를 발행한다. 센서들은 저마다 특유의 노이즈와 데이터 주기를 갖지만, ROS2 상에서 일관된 메시지 구조를 사용함으로써 센서 융합이나 제어 알고리즘 설계가 한층 편리하다.

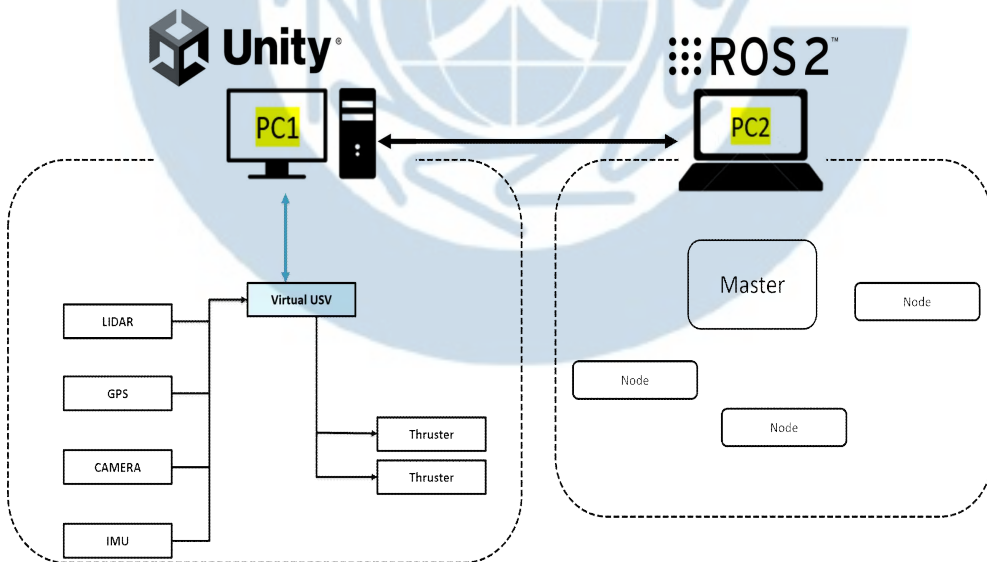


그림 4. ROS 통신 구성도

ROS2(Robot Operating System 2)는 분산 환경에서 노드 간 통신을 지원하

고, 센서·제어 알고리즘을 모듈 단위로 분리·관리할 수 있는 구조를 제공한다[18]. 본 연구에서는 ROS2의 노드(Node)·토픽(Topic)·서비스(Service) 개념을 적용하여, LIDAR·GPS·IMU·카메라 각 센서가 독립된 노드로 메시지를 발행하고, 자율항해 알고리즘 노드가 이를 구독하여 경로 계획이나 장애물 회피 등을 실행하도록 하였다. ROS2는 DDS(Data Distribution Service)를 기반으로 하여 QoS(Quality of Service) 설정을 통해 신뢰성·지연 등을 세밀하게 조정할 수 있으며, 이를 Unity 시뮬레이션과 실제 무인보트 간 통신에도 그대로 적용한다. Unity는 강력한 그래픽 엔진과 물리 엔진을 갖추고 있어, 보트의 움직임과 센서 신호를 가상으로 재현하기 용이하다. 본 연구에서는 Unity에서 Raycasting을 통해 가상 LIDAR 데이터를 생성하고, 가상 카메라를 통해 이미지를 만들어 ROS2 메시지로 발행한다. 이를 통해 시뮬레이션 단계에서 센서 데이터를 손쉽게 모사할 수 있고, 실제 센서 노드와 거의 동일한 구조로 코드가 재사용된다.

Unity와 ROS2를 연동하기 위해서는 Unity 측에서 ROS 메시지 타입(sensor_msgs, geometry_msgs 등)을 지원하고, ROS-TCP-Connector나 유사 패키지를 사용해 TCP 소켓으로 양방향 통신을 가능케 해야 한다. 가상환경에서 제어 알고리즘은 /cmd_vel 같은 토픽으로 모터 명령을 발행하고, 실제 보트에서는 동일한 토픽을 모터 드라이버가 구독해 추진력을 제어한다. 이런 방식으로, 시뮬레이션에서 개발한 코드가 무인보트 임베디드 환경에서도 큰 수정 없이 그대로 동작하도록 설계할 수 있다.

Ⅲ. 가상환경 구성 및 선박 제작

3.1 Unity 환경 구축

3.1.1 Unity 프로젝트 설정

무인보트의 가상 시뮬레이션 환경을 구축하기 위해 Unity 엔진을 사용하였다. Unity는 강력한 그래픽 처리 능력과 사용자 친화적인 개발 환경을 제공하여 복잡한 3D 시뮬레이션을 구현하는 데 적합하다[19]. 본 연구에서는 ROS2와의 호환성과 안정성을 고려하여 Unity 버전 2020.3 LTS(Long Term Support)를 선택하였다. 이 버전은 장기간의 지원과 업데이트를 제공하며, ROS2와의 연동에 필요한 패키지와 플러그인을 안정적으로 지원한다.

프로젝트 생성 과정은 다음과 같다. 먼저 Unity Hub를 실행하여 새로운 3D 프로젝트를 생성하고, 프로젝트 이름과 저장 경로를 지정하였다. 프로젝트 생성 후, Package Manager를 통해 필요한 패키지들을 설치하였다. ROS2와의 연동을 위해서는 ROS-TCP-Connector와 ROS-TCP-Endpoint 패키지가 필요하다. 이 패키지들은 Unity와 ROS2 간의 통신을 가능하게 하며, 메시지 타입의 자동 생성과 데이터 직렬화/역직렬화를 지원한다.

패키지 설치에 Unity의 Package Manager에서 Git URL을 통한 패키지 추가 방식으로 진행하였다. 예를 들어, ROS-TCP-Connector의 경우, Git URL로 <https://github.com/Unity-Technologies/ROS-TCP-Connector.git>를 입력하여 설치하였다. 설치 후에는 ROS 메시지 타입 생성을 위해 ROS 메시지 자동 생성 도구를 사용하여 프로젝트에 필요한 메시지 타입들을 생성하였다. 또한, 해양 환경의 사실적인 표현을 위해 Universal Render Pipeline(UR

P)을 적용하였으며, Shader Graph와 Visual Effect Graph 패키지를 추가로 설치하였다. 이러한 그래픽 패키지들은 물의 반사, 굴절, 파도 효과를 구현하는 데 사용되었다.

3.1.2 가상 해양 환경 구성

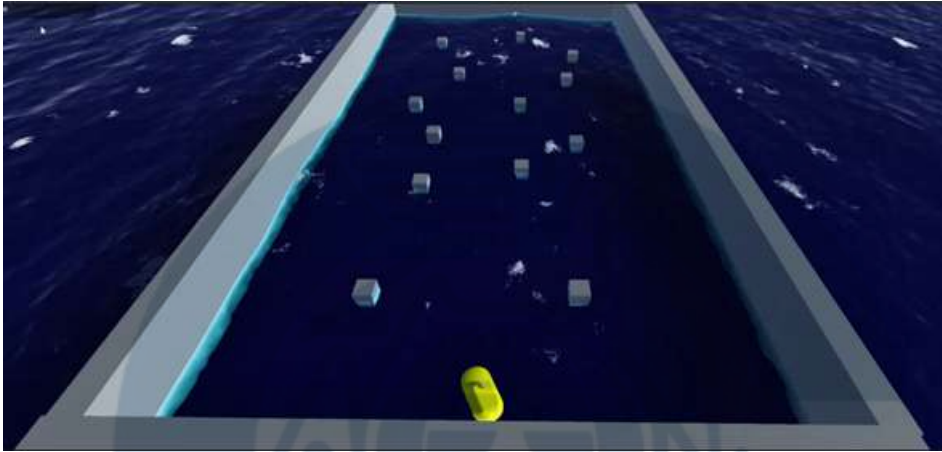


그림 5. Unity 가상 환경 화면

가상 해양 환경의 구축은 무인보트의 동작을 실제와 유사하게 시뮬레이션하기 위해 필수적이다. 해양 환경은 크게 수면 표현과 주변 요소 모델링으로 나뉜다.

수면 표현을 위해 Shader Graph를 활용하여 커스텀 셰이더를 제작하였다. 파도의 움직임은 Gerstner Wave 함수를 사용하여 사실적으로 표현하였다[19]. 또한, 가우시안 노이즈 함수를 추가하여 물결의 불규칙성을 구현하였고, Normal Map을 적용하여 수면의 세부적인 질감을 더하였다. 주변 환경 모델링은 Unity의 에셋 스토어에서 부표, 이정표, 등 적절한 모델을 가져와 배치하였다. 이러한 환경 요소들은 무인보트의 자율 항해 시나리오에 맞게 배치되었으며, 다양한 테스트 조건을 제공하기 위해 위치와 수량을 조절하였다.

3.2 가상 센서

3.2.1 LIDAR 가상 센서

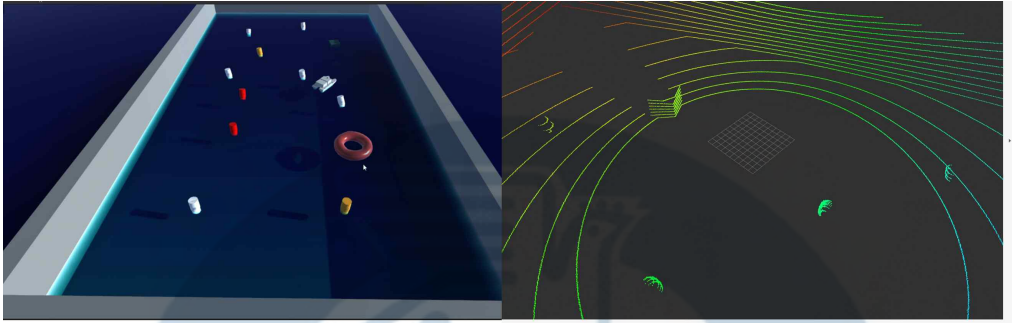


그림 6 Unity(좌)-ROS(우) 연결된 LIDAR 화면

LIDAR(Light Detection and Ranging) 센서는 주변 환경의 거리 정보를 획득하기 위한 핵심 장치이다. Unity에서 LIDAR 센서를 시뮬레이션하기 위해 Raycasting 기법을 사용하였다. Raycasting은 특정 방향으로 레이를 발사하여 물체와의 충돌 여부와 거리를 계산하는 방법으로, 실제 LIDAR의 레이저 광선 발사 원리를 모사할 수 있다[18].

본 연구에서는 Unity 엔진을 활용해 실제 보트의 LIDAR 센서를 가상으로 재현하였다. Unity 오브젝트(선박)를 기준으로, 0.5도 간격(또는 원하는 각도 해상도)으로 360도 범위를 아우르는 광선(Raycast)을 발사하여, 광선과 충돌한 오브젝트까지의 거리를 계산한다. 각 광선 레이의 충돌 지점까지의 거리를 배열 형태로 모아, ROS2의 LaserScan 메시지로 변환하고, 시뮬레이션 시간 스텝마다 발행한다. 이를 통해 실제 LIDAR처럼 주변 환경의 거리 정보를 얻을 수 있어, 장애물 회피 등 자율운항 알고리즘에서 동일한 프로세스를 사용할 수 있다. Unity 상에서는 RaycastHit 정보를 이용해 충

돌 여부와 거리를 획득하고, LaserScan 메시지의 `angle_increment`, `range_min`, `range_max` 등을 설정하여 실제 센서 모델과 유사한 출력을 만들었다.

3.2.2 가상 카메라

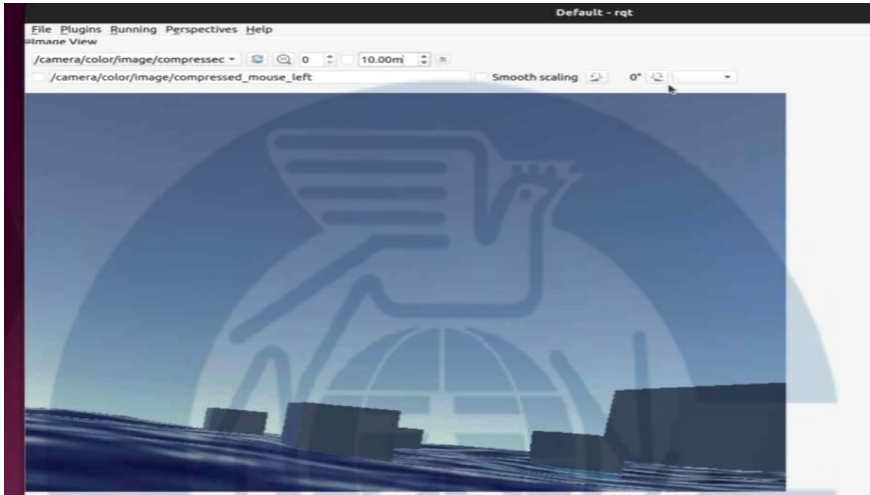


그림 7. Linux로 전송되고 있는 Unity의 가상카메라

시각 정보를 재현하기 위해서는 Unity 카메라를 배치하고, 해당 카메라 뷰를 ROS2의 Image 메시지 형태로 변환한다. 본 연구에서는 선박 오브젝트 전방에 카메라를 장착하고, RenderTexture를 사용해 카메라의 프레임 버퍼를 캡처한 뒤, RGB 이미지를 ROS2 Image 메시지로 직렬화하였다. 프레임레이트(FPS)와 해상도는 실제 보트의 카메라 스펙에 맞춰 30~60FPS, 640×480 혹은 1280×720 등으로 설정할 수 있으며, Unity 물리 엔진의 시간 흐름(Time.deltaTime)에 따라 영상 출력 주기를 제어할 수 있다. 이렇게 얻은 가상 카메라 영상은 객체 인식, 도킹 지점 탐지 등의 알고리즘에 직접 투입할 수 있으므로, 실제 환경 모델 검증에 미리 테스트하는 데 유용하다.

3.2.3 GPS 및 IMU 시뮬레이션

Unity에서는 오브젝트(선박)의 `transform.position`, `transform.rotation` 정보를 실시간으로 획득할 수 있는데, 이를 GPS와 IMU(Inertial Measurement Unit)의 데이터 형태로 매핑하였다. 예를 들어, 선박 오브젝트의 World 좌표(position)는 GPS가 제공하는 위도·경도·고도 값으로 변환 가능하며, rotation(roll, pitch, yaw)은 IMU가 측정하는 자세 값으로 대응시킬 수 있다. 최종적으로는 ROS2에서 `sensor_msgs/NavSatFix`, `sensor_msgs/Imu` 메시지를 발행하여, 실제 GPS·IMU 노드와 유사한 구조를 유지한다.

3.2.4 가상 쓰러스터

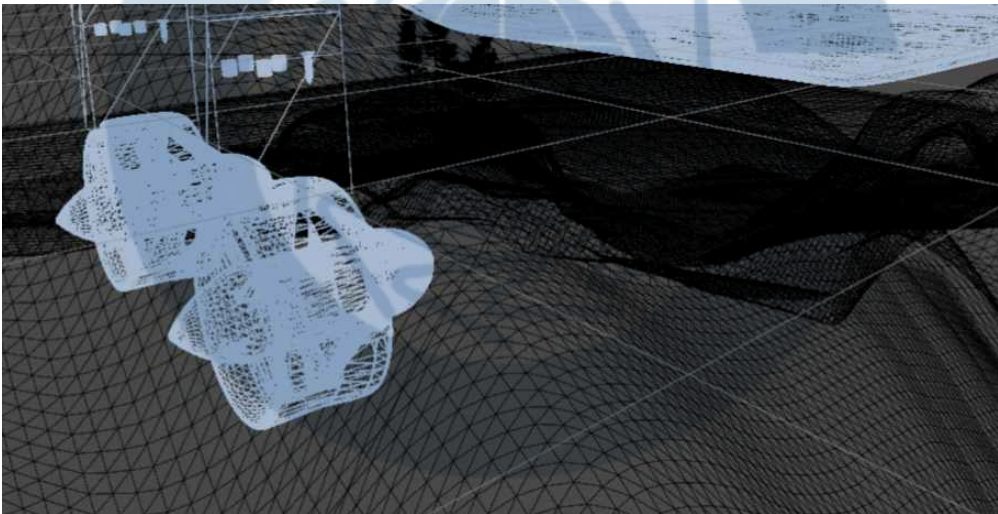


그림 8. Unity에서 구현된 가상 쓰러스터 모델

가상 선박은 Unity의 Rigidbody 컴포넌트를 통해 기본 물리 시뮬레이션을 수행하지만, 실제 보트처럼 좌우 쓰러스터 차등 제어로 움직이는 효과를 내기 위해선, `/cmd_vel(geometry_msgs/Twist)` 메시지를 받아서 두 개의 추력(thrust)을 가상으로 계산해야 한다. 예를 들어, 전·후진 속도(linea

r.x)와 회전 속도(angular.z)를 입력 받아, 좌우 모터에 가해지는 힘(force) · 토크(torque) 값을 결정한 뒤, Rigidbody.AddForce()와 AddTorque() 함수를 호출해 선박을 움직인다. 이 과정을 통해 선박은 회전 반경이나 관성 거동 면에서 실제 쓰러스터 방식과 유사한 반응을 보이게 된다. Unity 물리 엔진의 Drag, Angular Drag 등 파라미터를 적절히 조정하면, 수면에서의 저항 효과를 간단히 모사할 수도 있다.

3.2.5 센서 데이터 통합

여러 센서로부터 수집된 데이터의 동기화는 정확한 상태 추정을 위해 필수적이다. 모든 센서 데이터는 Unity의 시뮬레이션 시간에 기반하여 생성되며, ROS2 메시지의 header.stamp 필드에 동일한 타임스탬프를 부여하였다. 이를 통해 ROS2에서 센서 데이터를 수신할 때 시간적으로 정렬된 데이터를 처리할 수 있다.

센서 데이터의 퓨전(Fusion)은 ROS2에서 구현된 알고리즘을 통해 이루어졌다. 위치 추정을 위해 Extended Kalman Filter(EKF)를 사용하여 GPS와 IMU 데이터를 통합하였다. EKF는 센서 데이터의 노이즈 특성을 고려하여 보트의 위치와 속도를 추정하며, 이를 통해 보다 정확한 항법 정보를 제공한다.

센서 데이터의 통합 과정에서 각 센서의 시간 지연(Latency)과 데이터 속도(Rate)를 고려하였다. 필요한 경우 메시지 버퍼링이나 타임스탬프 보정을 통해 데이터의 동기화를 유지하였다. 이러한 센서 퓨전은 무인보트의 자율 항해와 장애물 회피 알고리즘의 성능을 향상시키는 데 중요한 역할을 한다.

3.2.6 알고리즘 구현 및 적용

무인보트의 자율 항해를 위한 제어 알고리즘은 ROS2 환경에서 구현되었으며, 경로 계획(Path Planning), 장애물 회피(Obstacle Avoidance), 위치 추정(Localization)과 같은 주요 기능을 포함한다. 이를 통해 자율 항해의 효율성을 높이고 다양한 환경에서 안정적으로 작동할 수 있도록 설계하였다.

경로 계획 알고리즘으로는 A* 알고리즘을 기반으로 한 전역 경로 계획(Global Path Planning)을 수행하였다. A* 알고리즘은 목표 지점까지의 최단 경로를 계산하며, 지도 정보와 장애물 위치를 통합하여 안전한 경로를 생성한다. 이 과정에서 LIDAR로 생성된 지도 데이터를 활용하여 탐색 공간을 구성하였으며, 경로의 비용 함수에는 장애물과의 거리, 경로의 곡률 등을 포함시켜 최적의 경로를 도출하였다.

LIDAR 센서에서 획득한 거리 데이터를 실시간으로 분석하여 장애물 회피 알고리즘을 구현하였다. 장애물 회피는 다음의 알고리즘들을 참고하여 설계되었다. VFH(Vector Field Histogram) 알고리즘을 활용하여 주변의 자유 공간을 탐색하고 이동 방향을 결정하였다. 이는 LIDAR 데이터를 기반으로 주변의 밀집도를 분석하고, 가장 적합한 회피 방향을 계산하는 방식이다[19]. DWA(Dynamic Window Approach)를 활용하여 로컬 경로 계획을 수행하였다[20]. DWA는 보트의 동역학을 고려하여 실시간으로 회피 경로를 생성하며, 특히 제한된 공간에서의 빠른 회피와 안정적인 이동에 유리하다.

포텐셜 필드 알고리즘(Potential Field Algorithm)을 적용하였으며, 장애물과의 거리 및 목표 지점과의 거리를 바탕으로 힘 벡터를 계산하여 이동 방향을 설정하였다[21]. 이 방식은 직관적이지만, 지역 최소점(Local Mini

ma) 문제를 방지하기 위해 수정된 형태로 적용하였다. 위치 추정은 GPS와 IMU 데이터를 통합하여 수행되었다.

앞서 설명한 EKF(Extended Kalman Filter)를 적용하여 센서 데이터의 노이즈를 감소시키고, 보트의 정확한 위치와 속도를 추정하였다. EKF는 GPS 데이터를 기반으로 전역 위치를 추정하고, IMU 데이터를 통해 자세 정보를 보정하였다. 제어 알고리즘에서는 PID 제어기를 사용하여 보트의 추진력과 방향을 조절하였다. 추정된 위치와 경로 계획에서 계산된 목표 경로를 비교하여 속도와 방향의 오차를 계산하고, 이를 보정하여 안정적인 항해를 유지하였다.

카메라를 활용한 객체 인식 알고리즘은 YOLOv8 모델을 기반으로 설계되었다. 이는 실시간으로 장애물을 인식하고 추적하며, 장애물의 이동 경로를 예측하여 회피 계획에 반영하였다. LIDAR와 카메라 데이터를 결합하여 객체의 3D 위치를 추정하는 방식을 적용하였으며, LIDAR 데이터의 포인트 클라우드와 카메라 영상의 특징점을 매칭하여 장애물의 정확한 위치를 파악하였다[22]. ROS2의 노드 기반 구조를 활용하여 알고리즘을 모듈화하였다. 각 기능은 독립적인 노드로 구현되었으며, 토픽 기반의 메시지 통신을 통해 데이터를 교환하였다.

Path Planning Node는 A* 알고리즘을, Obstacle Avoidance Node는 VFH, DWA, Potential Field Algorithm 기반의 회피 알고리즘을, Localization Node는 EKF를 통한 위치 추정을, Control Node는 PID 제어기를, Object Detection Node는 YOLOv8 기반 객체 인식을 각각 구현하였다. 본 연구에서 구현된 알고리즘은 LIDAR, GPS, IMU, 카메라 데이터를 통합하여 자율 항해를 위한 안정적이고 효율적인 제어를 가능하게 하였다.

3.3 3D 프린터 선박 제작 및 임베디드 시스템 설치

3.3.1 선체 모델링(Rhino) 및 출력

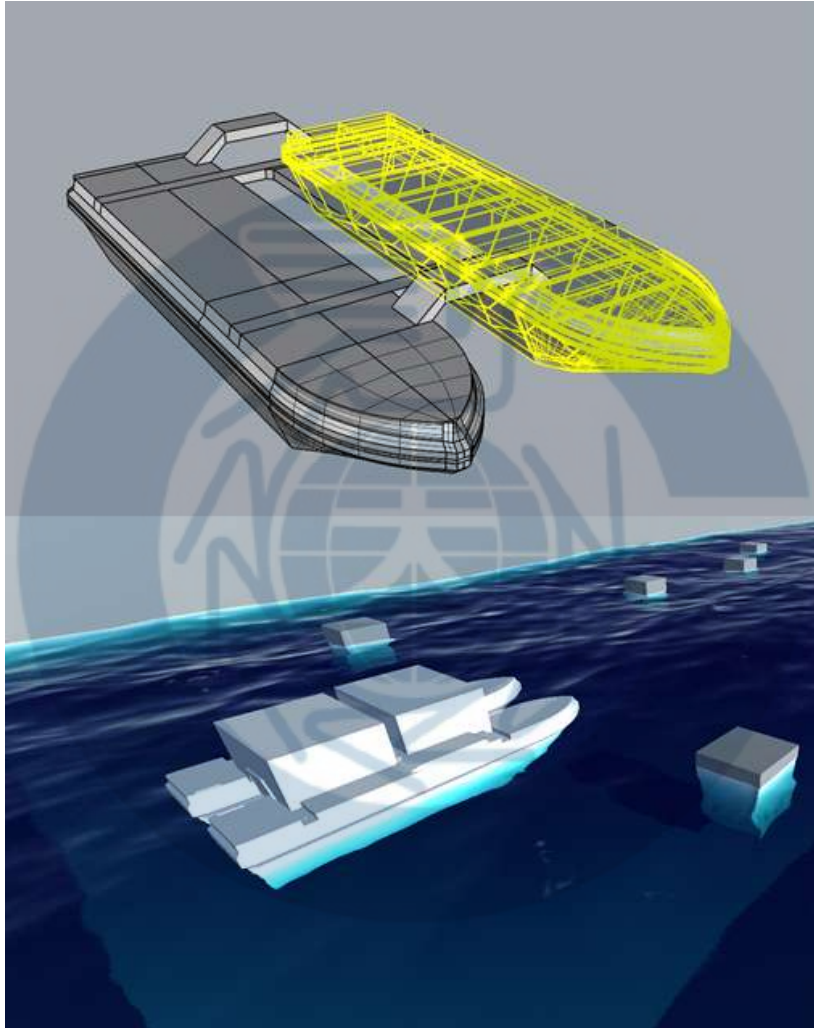


그림 9. Rhino에서 제작된 무인보트(위)와 Unity 적용(아래)

본 연구는 쌍동형구조를 채택하였으며, Rhino 등 3D 모델링 소프트웨어를 활용해 선체 형상을 설계하였다. 쌍동선은 양쪽에 두 개의 좁은 선체가 배

치되어 부유 안정성과 선회 성능을 동시에 확보할 수 있다는 장점이 있다. Rhino로 모델링한 결과물을 FBX 형태로 내보내 Unity에 배치함으로써, 가상환경에서 시각적으로도 동일한 선체를 재현하였다. 이렇게 가상 선체와 실제 선체가 유사한 지오메트리를 공유하면, 시뮬레이션에서 발생할 수 있는 형상 차이로 인한 오차를 어느 정도 줄일 수 있다. 설계가 완료된 모델은 3D 프린터로 출력한다. 쌍동선 구조이므로 좌·우 선체를 각각 출력하거나, 크기에 따라 여러 부품으로 나눈 뒤 결합할 수도 있다.

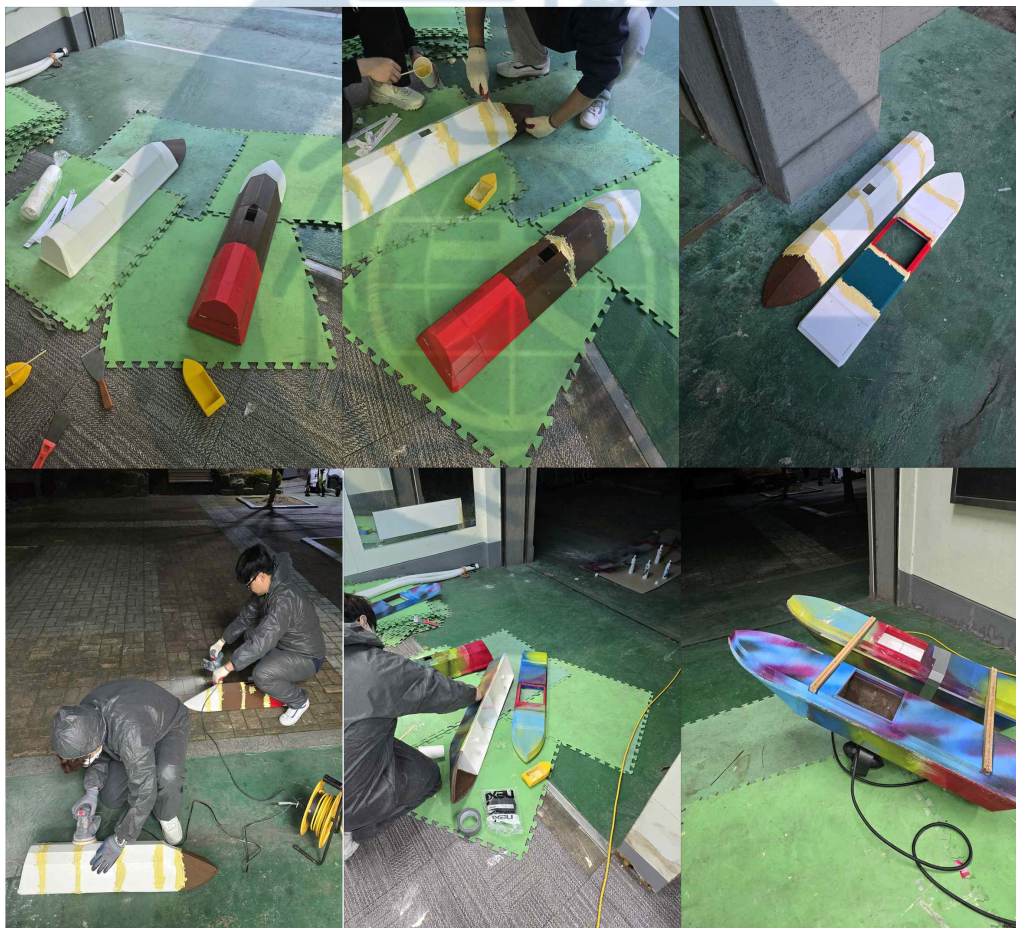


그림 10. 무인선박 제작 과정

출력 후에는 지지대(서포트) 제거, 샌딩(sanding), 표면 정리 과정을 거쳐 조립한다. 각 부품 접합부를 수지·에폭시 등으로 단단히 접착하고, 방수 코팅 작업을 꼼꼼히 시행하여 내부에 물이 스며들지 않도록 한다. 필요하다면 선체 내부에 발포 스티로폼이나 부력재를 추가해 침수 위험을 낮출 수 있다. 쌍동선 중앙을 연결하는 브릿지(bridge) 파트 역시 별도 출력 후 프로파일러로 보강하여, 두 선체 간 결속력을 높이도록 설계했다.

3.3.2 하드웨어 배치

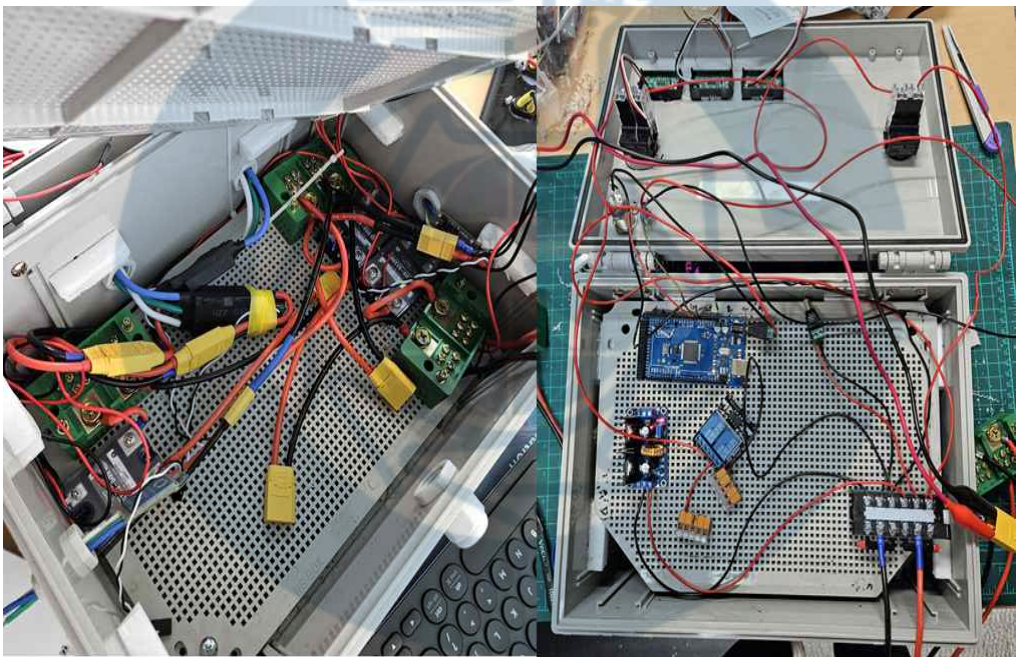
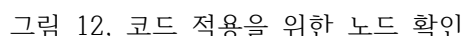


그림 11. 컨트롤 박스 제작 과정

완성된 선체 내부에는 젯슨 나노(Jetson Nano) 보드, ESC(전자속도제어기), 배터리, LIDAR·GPS·IMU·카메라 등을 장착한다. U센서들은 실제 환경에서 필요한 위치와 시야각을 고려해 설치한다. 예컨대 LIDAR는 시야가 막히지 않는 지점(선체 최상단 혹은 전방 상단)에 부착하여, 360도 장애물

전기 배선을 완료한 뒤, 모터 드라이버(ESC)와 젯슨 나노 간에는 `/cmd_vel` 같은 토픽을 주고받을 수 있도록 한다. Unity 시뮬레이션에서 `/cmd_vel` → 가상 스러스터로 mapping하던 구조가, 실제 환경에서는 `/cmd_vel` → ESC PWM 신호로 변환되어 모터를 구동한다. 이를 위해 ROS2의 `geometry_msgs/Twist` 메시지를 받아서, 좌우 모터에 할당할 추진력(스러스터 출력을 변환한 PWM 값)을 계산하는 노드를 작성하거나, 간단한 PID 제어 로직을 추가할 수도 있다. 가상환경 단계에서 개발된 코드가 임베디드 환경에서도 동일하게 빌드·실행 가능하도록, 센서와 드라이버 노드의 이름, 메시지 타입, QoS 설정 등을 일치시킨다.



코드 적용 절차는 크게 세 단계로 요약된다.

첫째, Unity에서 작성된 ROS2 패키지를 젯슨 나노로 복사하고, 시뮬레이션 용 노드(가상 센서, 가상 쓰러스터)를 제외한 실제 센서 드라이버 노드만 활성화하도록 launch 파일을 수정한다.

둘째, 각 센서의 드라이버 설치와 파라미터 설정을 맞춘 뒤, `ros2 run` 혹은 `ros2 launch` 명령을 통해 노드를 구동한다.

셋째, 모터 동작 여부와 센서 토픽을 `rviz`, `rqt` 등 툴로 모니터링하면서, 코드가 예상대로 작동하는지 확인한다. 여기서 기기 간 네트워크 설정(와이파이 사용 시 IP 대역, DDS 설정 등)이 Unity 환경과 다를 수 있으므로, `ROS_DOMAIN_ID`나 `multicast` 주소 등이 충돌하지 않도록 주의한다.

이러한 과정을 마치면, Unity 시뮬레이션에서 개발해온 자율항해 로직(예: GPS 주행, LIDAR 장애물 회피, 카메라 도킹 등)을 실제 무인보트에서도 큰 수정 없이 테스트할 수 있게 된다. 다시 말해, 시뮬레이션 단계에서 수행한 센서 메시지 처리가 물리 센서에 대한 드라이버로 그대로 대체될 뿐, 알고리즘 본체와 토픽 구조는 동일하게 유지된다는 점이 본 연구의 핵심이다. 실제 물 위 실험 전에도 `rosbag`으로 센서 데이터를 기록하여 디버깅하거나, 짧은 수조 테스트 후 즉각 코드 수정을 진행할 수 있어, 개발 효율이 크게 향상된다.

3D 프린터 선체 조립과 전장 시스템 연결, 그리고 Unity 시뮬레이션 코드의 임베디드 이식 단계를 통해, 가상환경-현실환경 간 자율항해 소프트웨어가 일관성 있게 동작하는 기반이 마련된다. 이는 자율운항 알고리즘을 실제 보트 위에서 검증하기 전에 여러 번 가상환경에서 실험함으로써, 비용과 위험 부담을 최소화할 수 있다는 의의를 갖는다.

IV. 임베디드 시스템 코드 테스트

4.1 테스트 환경 설정 및 실행



그림 13. 실험을 위한 수조

필드 테스트는 무인보트의 실제 운용 성능을 평가하기 위해 가로 25미터, 세로 15미터 규모의 수조에서 진행되었다. 각 시나리오는 Unity 시뮬레이션 단계에서 이미 코드 구조를 확인한 뒤, 임베디드 보드 실제 센서를 결합해 물 위에서 재현하는 것을 목표로 한다. 임베디드 보드(Jetson Nano)에 Ubuntu 22.04와 ROS2 Humble을 설치하고, LIDAR · GPS · IMU · 카메라 등 실제 센서의 드라이버를 준비한다. 시뮬레이션용 ROS2 패키지를 그대로 복사하되, 가상 센서 노드(예: Unity Raycast, 가상 카메라) 대신 실제 센서 드라이버 노드가 활성화되도록 launch 파일을 조정한다. 네 가지 테스트 시나리오에 맞춰, 각각의 노드(예: Localization Node, Obstacle Avoidance Node, Object Detection Node 등)도 분기별로 실행 또는 launch 파일

파라미터를 변경하면, 시뮬레이션 때와 동일한 논리 흐름을 유지한 채 물리 센서 데이터를 받아 제어 명령을 발행할 수 있다.

GPS 주행의 경우, 수조나 실험 장소가 협소할 수 있으므로 실험 가능 범위 내에서 웨이포인트를 설정한다. LIDAR 장애물 회피 시에는 장애물 배치 위치와 크기를 달리하며, 회피 알고리즘이 제대로 작동하는지 관찰한다. 이미지 인식 도킹의 경우, 수조 한쪽에 도형(예: 빨간색 보드나 QR 코드)를 세워 놓고, 카메라가 이를 인식해 /cmd_vel를 통해 천천히 접근하도록 구현한다. 종합 과제는 여러 노드가 동시에 움직여 예기치 못한 충돌이 없는지 확인해야 하므로, QoS와 센서 주기, 제어 우선순위를 면밀히 살펴본다.

시나리오	목표	주요 센서	기능 요약
GPS 기반 웨이포인트 주행	미리 설정된 GPS 좌표(웨이포인트)까지 무인보트가 자율 이동	GPS, IMU	GPS 데이터를 통해 보트의 현재 위치를 파악하고, IMU로 자세를 보정하며 /cmd_vel 메시지로 추진력·회전 명령을 수행
LIDAR 장애물 회피	전방 혹은 측면에 놓인 부표·상자 등 장애물을 실시간으로 피하기	LIDAR	/scan 토픽의 거리 데이터를 분석해 장애물이 임계거리 이내로 들어오면 회피 명령을 발행
이미지 인식을 통한 도킹	카메라 영상에서 특정 도형(색상·패턴 등)을 인식하고, 해당 지점으로 접근해 정밀 정박(도킹)	카메라	/image 토픽에 대한 객체 인식 노드가 도형 위치를 계산하여 /cmd_vel로 미세 조종을 수행
통합 과제 수행	GPS 주행, 장애물 회피, 이미지 인식 도킹을 병행하여 종합 임무를 완수	GPS, LIDAR, 카메라, IMU 등	여러 노드가 동시에 구동되며, 상황에 따라 회피 로직과 도킹 로직을 우선 적용

표 1. 테스트를 위한 시나리오

표 1은 무인보트의 자율항해 기능을 실험하기 위한 주요 시나리오를 요약한 것으로, 각 시나리오의 목표, 사용 센서, 그리고 기능 요약을 정리하여 실험의 목적과 접근 방식을 정리하였다. 이러한 실험은 GPS 기반 웨이포인트 주행, LIDAR를 이용한 장애물 회피, 이미지 인식을 통한 도킹, 그리고 이를 종합적으로 수행하는 통합 과제 등 네 가지 주요 실험으로 나뉜다.

4.2 테스트 결과 및 분석

아래 표는 본 연구에서 수행한 네 가지 임무 시나리오(웨이포인트 주행, 장애물 회피, 이미지 센싱, 통합 임무)의 성공·실패 횟수와 주요 원인을 요약한 것이다. 각 시나리오는 총 10회씩 반복하였으며, 실험 조건과 성공 판정 기준은 시나리오별로 조금씩 달랐다.

시나리오	성공/시도	성공 판정 기준	주요 실패 원인
웨이포인트 주행	09/10	경로에서 3m 이상이탈하지 않고 최종 목표점 도달	GPS 기기 오류, 위치값 급변, 오차 발생
장애물 회피	8/10	4개 장애물 전부 충돌 없이 회피	센서 노이즈, 선박 움직임 불안정, 부딪힘
이미지 센싱(도킹)	9/10	목표 이미지 앞 1m 이내 접근	빛 반사로 목표 인식 실패, 단순 직진
통합 임무(종합 시나리오)	7/10	목표 지점 4곳 순차 방문 + 장애물 회피 + 최종 도킹	노드 간 충돌(동시 명령), 센서 간 간섭 등

표 2. 테스트 결과

웨이포인트 주행

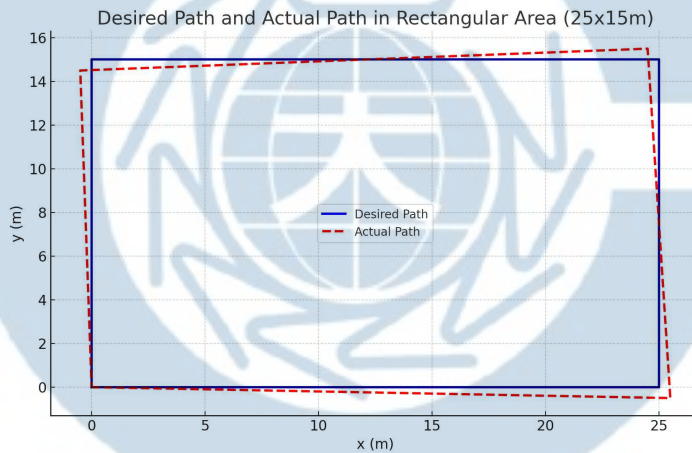


그림 14. 웨이포인트 주행(위)와 주행 경로 비교(아래)

총 10번 시도 중 9번 성공하였으며, 1번은 GPS 기기가 알 수 없는 오류로 인해 일시적으로 부정확한 위치 값을 보내면서 보트가 경로에서 크게 벗어났다. 해당 실험에서는 경로 계획에서 3미터 이상 이탈하면 실패로 간주하였다. 야외 환경에서 GPS가 갑자기 급변하는 현상이 관찰되었는데, 이는 위성 신호 문제나 일시적인 데이터 간섭 등이 원인으로 추정된다.

장애물 회피

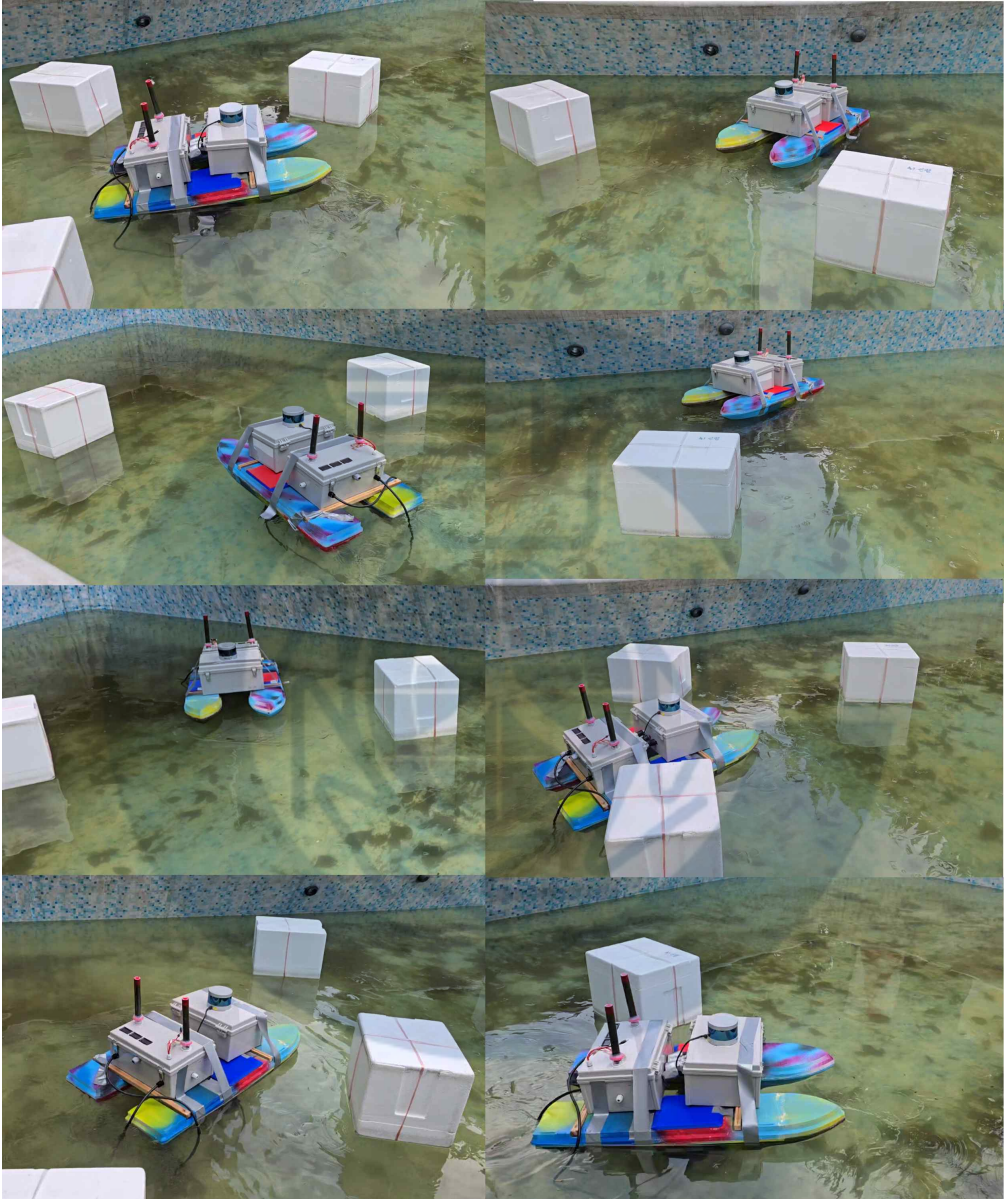


그림 15. 무인보트의 장애물 회피

장애물은 4개를 임의의 위치에 배치하고, LIDAR 데이터를 바탕으로 충돌 없이 4개 모두 회피한 경우를 성공으로 간주하였다. 10번 중 8번이 성공하

였고, 2번은 물체에 직접 부딪혔다. 실제 센서에서 발생하는 노이즈(거리 값 흔들림)와 보트의 회전 관성으로 인해, 회피 명령이 늦거나 충분히 크게 꺾지 못해 접촉이 일어났다. 좁은 수조 환경에서 회피 기동 시 여유 폭이 적은 점도 한 요인이었다.

이미지 센싱(도킹)



그림 16. YOLO 적용(위)와 도킹 하는 장면

카메라 영상을 통해 특정 도형을 감지하고, 그 앞 1m 이내로 접근하면 성공으로 간주했다. 총 10회 중 9번 성공하였으며, 1번은 수면 반사광(빛 반사) 때문에 카메라가 도형 잃어버려, 단순 직진하다가 목표를 놓쳤다. 실제 환경에서는 조명 변화나 물 반사광으로 인해 인식률이 떨어지는 경우가 있어, 시뮬레이션보다 보정 파라미터가 필요한 것으로 보인다.

통합 임무(종합 시나리오)

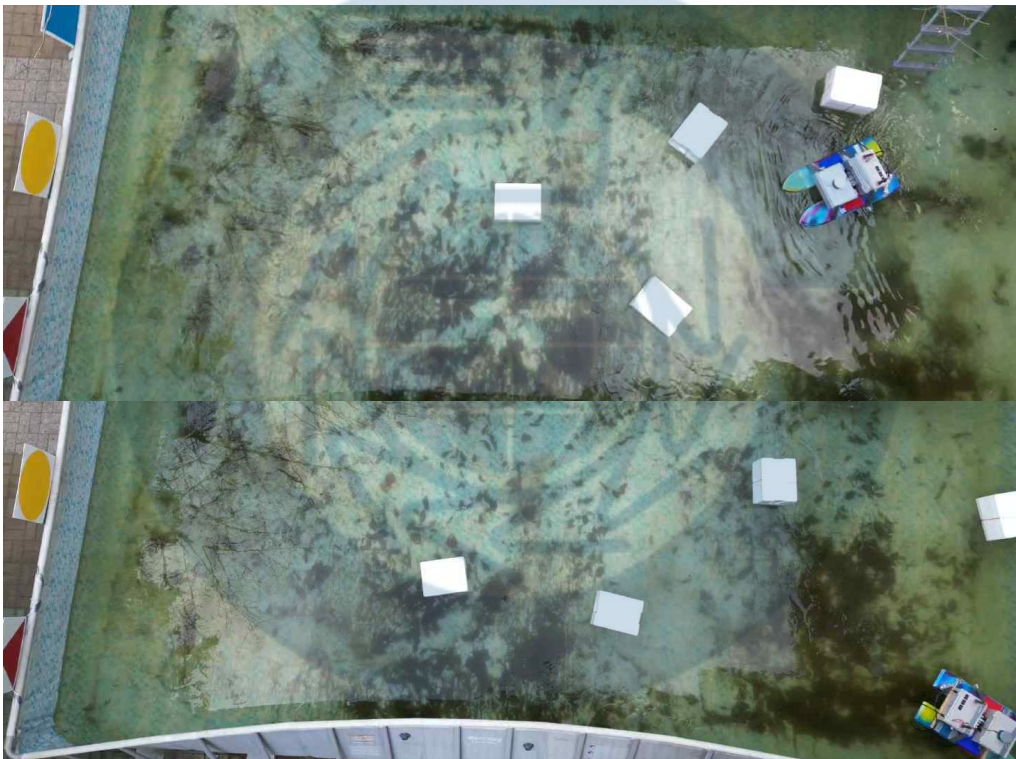


그림 17. 종합 임무 시나리오

통합 임무는 GPS 웨이포인트를 4곳 순차 방문, 그 과정에서 장애물이 나타나면 회피 알고리즘을 우선 수행하며, 최종 구간에서 카메라 도형을 인식해 도킹하는 흐름으로 구성하였다. 10회 시도 중 7번이 성공하였고, 나머지 3번은 중간에 충돌하거나 도킹 시점을 혼동했다. 실패 원인을 살펴보

면, “노드 간 충돌” 즉, 장애물 회피 노드가 움직이던 중 갑작스럽게 이미지 인식 노드가 도킹 명령을 내리면서 보트가 엉뚱한 방향으로 꺾인 사례가 있었다. 또한 센서 간 간섭이나 메시지 우선순위가 제대로 설정되지 않아, 한쪽 노드에서 /cmd_vel을 계속 덮어쓰는 문제가 발생하기도 했다. 이런 사례는 멀티 노드가 동시에 메시지를 발행할 때 각 알고리즘 간 협조가 필수적임을 시사한다.

위와 같이 각 임무를 별도로 수행할 때(웨이포인트 주행, 장애물 회피, 이미지 센싱)는 80~90% 수준의 성공률을 보였으나, 복합 기능을 동시에 요구하는 “통합 임무”에서는 센서 노이즈와 노드 우선순위 충돌 문제가 겹치며 성공률이 약간 떨어졌다. 이는 시뮬레이션에서 발견하지 못한 현실 세계의 노드 상호작용 이슈를 잘 보여주는데, 추후에는 토폴 중재나 노드 간 상태 공유 기법을 추가해 개선할 수 있을 것으로 보인다.

결론적으로, 본 연구의 시뮬레이션 환경에서 작성된 코드는 실제 무인보트에서도 기본 임무를 상당히 높은 성공률로 수행할 수 있음을 확인하였으며, 센서 노이즈나 노드 충돌과 같은 현실적 문제를 부분적으로나마 식별·보완할 수 있었다. 이는 Unity-ROS2-실제 임베디드 간 통합 접근법이 실용적으로 유효함을 보여준다.

V. 결론

본 연구에서는 Unity 기반 가상환경과 ROS2를 결합하여, 무인보트 자율항해 알고리즘을 시뮬레이션 단계에서 빠르게 개발·디버깅한 뒤, 동일한 코드 구조를 임베디드 시스템(젯슨 나노)과 실제 소형 무인보트에 적용하는 과정을 제시하였다. 기존 연구사례 분석을 통해 쌍동형 선체를 3D 프린터로 제작하고, LIDAR·GPS·IMU·카메라 등 센서들을 배치하여 수조 환경에서 테스트할 수 있는 무인보트를 구성하였다. 또한 Unity에서는 Raycasting을 이용한 가상 LIDAR, position·rotation 기반 GPS·IMU, 카메라 RenderTexture 등 가상 센서 구현을 통해 실제 환경과 최대한 유사한 토픽 구조를 유지하였으며, 가상 쓰레스터로 선박 움직임을 모사하여 /cmd_vel 등 ROS2 명령을 동일하게 처리하도록 했다.

시뮬레이션과 현실 환경에서의 소프트웨어 호환성은, ROS2 노드 및 메시지 구조를 통일함으로써 크게 향상되었다. 실제 수조 환경에서 GPS 기반 웨이포인트 주행, LIDAR 장애물 회피, 카메라 인식 도킹, 그리고 종합 임무 수행 등을 시도했을 때, Unity에서 작성한 코드가 별다른 수정 없이 임베디드 시스템 위에서 동작하는 모습을 확인하였다. 다만 수조 규모와 센서 노이즈, 하드웨어 응답 지연 등 현실적인 제약 요인으로 인해 시뮬레이션 대비 성능 편차가 나타났으며, 시뮬레이션에서 간과된 물리 현상(노이즈, 반사파, 빛)으로 인해 장애물 회피나 도킹 성공률이 다소 낮아지는 경우도 관찰되었다.

그럼에도 불구하고, 본 연구의 접근법은 무인보트 개발 초기 단계에서 개발 비용과 위험 부담을 낮추고, 반복 실험과 디버깅을 안전하게 진행할 수 있는 방법론으로서 의의를 가진다. Unity의 강력한 그래픽·물리 엔진을 활용해 다양한 시나리오와 센서 모델을 쉽게 구성할 수 있고, ROS2 토픽을

동일하게 재사용함으로써 시뮬레이션과 현실 환경이 유기적으로 연결되는 통합 프레임워크를 구축할 수 있다. 향후에는 딥러닝 기반 객체 인식·추적 알고리즘을 도킹 등 자율운항 전 과정에 적극 도입해, 복잡한 해양 환경에서도 안정적으로 동작하는 다양한 무인보트 시스템을 구현할 수 있을 것이다. 본 연구에서 제시한 Unity-ROS2-실제 보트 연동 기법은 중소형 연구팀이 무인보트 프로토타이핑을 수행할 때, 시뮬레이션과 현실 간 간극을 줄이고 개발 효율을 높이는 데 기여할 것으로 기대된다.



참고문헌

- [1] 이데일리, 박순엽. (2021.11.29), “바다 위 테슬라’ 자율운항 선박…글로벌 조선업계 개발 박차” , <https://www.edaily.co.kr/news/read?newsId=03834326629250640&mediaCodeNo=257&OutLnkChk=Y>.
- [2] 해양수산부. (2020). “바다의 미래를 주도할 자율운항선박 개발 프로젝트 본격착수” , https://www.mof.go.kr/synap/view.do?fn=MOF_ARTICLE_32722_20200617172c0c7428a540&fd=202211.
- [3] Kim, Y.J., Kim, J.S., Lee, S.Y., Min, K.W., 2018, Implementation of Control System for Autonomous Vehicle Using Gazebo and ROS, The I EIE Fall Conference 2018, pp. 621-624.
- [4] Freese, Marc, et al. "Virtual robot experimentation platform v-rep: a versatile 3d robot simulator." Simulation, Modeling, and Programming for Autonomous Robots: Second International Conference, SIMPAR 2010, Darmstadt, Germany, November 15-18, 2010. Proceedings 2. Springer Berlin Heidelberg, 2010.
- [5] 비즈커스, 임영무 (2024.08.19), “HD현대 아비커스 개발자가 말하는 해양모빌리티의 미래” , <https://news.tf.co.kr/read/economy/2125704.htm>.
- [6] Hussein, A., García, F., & Olaverri-Monreal, C. (2018). "ROS and unity-based framework for intelligent vehicle control and simulation." In ICVES International Conference on Vehicular Electronics and Safety, Madrid, Spain, pp. 12-14.

- [7] Hong, S.K. (2015). "Analysis of factors influencing ship collision avoidance judgment of maritime officers." *Journal of the Korean Institute of Plant Engineering*, 20(1), pp. 3-10.
- [8] Jo, H.J., Kim, J.H., Kim, S.R., Woo, J.H., & Park, J.Y. (2021). "Development of autonomous algorithm for boat using robot operating system." *Journal of the Society of Naval Architects of Korea*, 58(2), pp. 121-128.
- [9] Chen, S., Liu, B., Feng, C., Vallespi-Gonzalez, C., & Wellington, C. (2021). "3D point cloud processing and learning for autonomous driving: Impacting map creation, localization, and perception." *IEEE Signal Processing Magazine*, 38(1), pp. 68-86.
- [10] Jo, Y.S., Jung, H.J., Lee, S.H., & Jeong, D.W. (2022). "LiDAR sensor and pure pursuit algorithm-based autonomous navigation system for small ships." *Journal of Korean Institute of Information Technology*, 20(4), pp. 1-11.
- [11] Fossen, Thor I. (2011). *Handbook of marine craft hydrodynamics and motion control*. John Wiley and Sons.
- [12] Matsumoto, N., & Suemitsu, K. (1983). "Interference effects between hull, propeller, and rudder of a hydrodynamic mathematical model in maneuvering motion." *Journal of Kansai Society of Naval Architects*, 93, pp. 45-62.
- [13] Kim, K.W., Kim, H.Y., Choi, S.E., Na, K.I., Lee, H., & Seo, J. W. (2022). "Development of ship dynamics model by free-running model tests and regression." *Journal of the Society of Naval Architects of Korea*, 59(3), pp. 173-182.

- [14] Lee, E.J., Ruy, W.S., & Seo, J. (2020). "Application of reinforcement learning to fire suppression system of an autonomous ship in irregular waves." *International Journal of Naval Architecture and Ocean Engineering*, 12, pp. 910-917.
- [15] Son, D.G., & Kim, D.H. (2021). "Autonomous navigation of unmanned surface vehicle based on fuzzy control considering COLREG." *Journal of Korean Institute of Intelligent Systems*, 31(1), pp. 11-20.
- [16] Cashmore, Michael, et al. (2015). "Rosplan: Planning in the robot operating system." *Proceedings of the International Conference on Automated Planning and Scheduling*, vol. 25.
- [17] L. Ecole, W.T. Kim, J.S. Yoon, et al. (2023). "Unity: A powerful tool for 3D computer animation production." *Journal of the Korea Computer Graphics Society*, vol. 29, no. 3, pp. 45-57.
- [18] Pandey, Jyotsna, & Hasegawa, Kazuhiko. (2015). "Study on maneuverability and control of an autonomous Wave Adaptive Modular Vessel (WAM-V) for ocean observation." *2015 International Association of Institutes of Navigation World Congress (IAIN)*, IEEE.
- [19] Borenstein, J., & Koren, Y. (1991). "The vector field histogram-fast obstacle avoidance for mobile robots." *IEEE Transactions on Robotics and Automation*.
- [20] Fox, D., Burgard, W., & Thrun, S. (1997). "The dynamic window approach to collision avoidance." *IEEE Robotics and Automation Magazine*.
- [21] Khatib, O. (1986). "Real-time obstacle avoidance for manipulators and mobile robots." *The International Journal of Robotics Research*

h.

[22] Zhang, H., Zhao, C., & Liu, Y. (2020). "Fusion of LiDAR and camera data for object detection in autonomous vehicles." IEEE Transactions on Intelligent Transportation Systems.



ABSTRACT

Implementation of a Small Unmanned Surface Vehicle Embedded System Using a Unity-ROS-Based Virtual Simulation Environment

LEE SEJIN

Dept. Culture and Technology Convergence
Graduate School of Changwon National University

Recent advances in unmanned surface vehicle (USV) technology have drawn considerable attention for applications such as marine monitoring, search-and-rescue, and safety management. However, the development and testing of USVs still face high costs and physical constraints. To address this challenge, we propose a method that combines a Unity-based virtual environment with ROS2 to rapidly implement and verify USV autonomous navigation algorithms in a simulation stage, and then seamlessly deploy the same code structure on a real embedded system.

Initially, we surveyed previous studies to identify a suitable catamaran-type hull for a small USV, and to determine the arrangement of sensors including LIDAR, camera, GPS, and IMU. In the Unity environment, we implemented virtual sensors such as LIDAR (via raycasting), GPS-IMU emulation (via transform data), and camera imaging (via RenderTexture) to publish ROS2 messages. This approach facilitates easy debugging and refinement of the code in simulation,

and allows for direct application of the same ROS2 nodes on an actual USV equipped with a 3D-printed hull, onboard sensors, and a Jetson Nano board.

Our tank-based experiments showed that individual tasks—such as GPS-based waypoint navigation, LIDAR-based obstacle avoidance, and camera-driven docking—achieved success rates of around 80-90%. In a combined mission scenario, the success rate slightly dropped due to node conflicts and sensor noise, highlighting the gaps between simulation and real-world conditions. Nonetheless, our results confirm that a consistent ROS2 framework bridging virtual and physical environments can significantly reduce initial development risks and costs, enabling safe iterative testing. Future work includes extending the system to larger vessels, expanding open-water experiments, and integrating more robust deep-learning-based perception to further enhance the versatility of USV technology.

Keywords

USV, Autonomous Navigation, ROS2, Unity Simulation, 3D Printing, Virtual simulation

감사의 글

먼저 부족한 논문을 심사해 주신 남상훈, 오범석, 유선진 교수님께 진심으로 감사드립니다. 바쁘신 와중에도 귀한 시간을 내어 꼼꼼히 검토해 주시고, 발전적 조언을 아끼지 않으신 덕분에 미흡한 점을 보완하고 논문을 완성할 수 있었습니다.

특히 지도교수 유선진 교수님께 깊은 감사의 말씀을 드립니다. 어디로 가야 할지 방황하던 저를 흔쾌히 받아 주시고, 지난 2년 동안 가까이에서 지켜보며 열정적인 연구 자세와 끊임없는 도전 정신을 몸소 보여주셨습니다. 교수님의 높은 기대에 미치지 못한 점이 많아 죄송하지만, 이 기간 동안 교수님께 보여주신 열정과 긍정적인 태도는 앞으로의 삶에도 큰 동력이 될 것 같습니다. 교수님처럼 뜨거운 열정으로 배움을 이어가는 사람이 되고 싶습니다.

함께 생활하며 연구실 생활을 풍부하게 만들어 준 공규형, 현빈이, 인주, Ruben, 하람이, 철우, 채림이 모든 연구실 식구들에게도 감사의 마음을 전합니다. 비록 제가 먼저 떠나지만, 모두가 남은 연구와 학위 과정을 잘 마무리하길 진심으로 응원합니다.

또한, 동현이 형에게는 특히 감사하다는 말씀을 전하고 싶습니다. 추운 겨울에 호수에 나가 실험하고, 밤새 함께 논문을 쓰며 2년간 온갖 어려움을 헤쳐 나갈 수 있었던 건 형이 없었다면 불가능했을 것입니다. 말도 잘 안 듣지 않는 못난 동생이었음에도 끝까지 믿어 주시고, 종종 일어난 사고도 잘 처리해 주신 덕분에 여기까지 올 수 있었습니다.

마지막으로 언제나 저를 믿고 응원해 주시는 가족들과, 힘들 때마다 귀 기울여 주고 도움을 아끼지 않았던 친구 윤종운에게도 감사의 인사를 드립니다. 제가 받은 사랑을 더크게 돌려 줄 줄 아는 사람으로 거듭나겠습니다. 감사합니다